Movie Discovery, Review & Cinema Booking Database

Oracle test report - full run of all project scripts | 2026-10-01 21:51:39

Database	Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production Version 21.3.0.0.0
Connection	XEPDB1 pluggable database, schema MOVIEDB (local Oracle XE install)
Client	SQL*Plus 21c, scripts run in order 02 -> 03 -> 04 -> 05 -> 06, each in one session
Run started / finished	2026-10-01 21:51:39 / 2026-10-01 21:51:43

OVERALL RESULT: PASS - every script ran and every check succeeded.

Script	Size	What was verified	Result
02_create_tables.sql	19 statements	15 tables + 2 sequences created	PASS
03_sample_data.sql	202 statements	199 INSERTs + 84 seats via PL/SQL; counts match	PASS
04_queries.sql	37 queries	37/37 correct (29 value-checked, all row counts cross-checked)	PASS
05_views.sql	5 views + 6 demo queries	all views created; demo output correct	PASS
06_booking_integrity.sql	23 statements	11 invalid operations rejected; savepoint/rollback correct	PASS

How results were checked

Every statement was executed against a real Oracle 21c database. For each query, the output was compared with the **Expected:** answer written in the script comments. Queries without a written expectation were checked against answers worked out independently from the sample data. Every query's row count was also cross-checked against a separate re-implementation of all 37 queries. For the integrity script, each statement marked **EXPECT ERROR** must fail with exactly that Oracle error code and constraint name, and every other statement must succeed.

Issues found and fixed before this run

Issue	Effect	Fix applied
03_sample_data.sql: several INSERTs on one line (12 movie_genre lines)	SQL*Plus rejected them with ORA-00933, so 28 genre links were not loaded	One statement per line
03 / 06: a '-- comment' after the ';' on 15 lines	SQL*Plus does not see the ';' as the end of the statement, so those rows were lost (ORA-00933)	Comment moved before the ';' as /* ... */

These fixes do not change any SQL logic. The scripts also still run in SQL Developer. Minor notes, not errors: Q35's comment names the user (ananya_m) but the query returns the user_id (6). Q14 has a tie, so the order of rahul_v and karthik_n may vary.

Step 5 - 02_create_tables.sql

19 statements, 0 errors. The drop/cleanup PL/SQL block ran, then 15 tables and 2 sequences (booking_seq, payment_seq) were created. Result: **PASS**

Tables in USER_TABLES
ACTOR
APP_USER
BOOKING
BOOKING_SEAT
GENRE
MOVIE
MOVIE_ACTOR
MOVIE_GENRE
MOVIE_SHOW
PAYMENT
REVIEW
SCREEN
SEAT
THEATRE
WATCHLIST

Step 6 - 03_sample_data.sql

199 INSERT statements succeeded, plus 84 seats generated by the PL/SQL loop; 0 errors; COMMIT complete. Result: **PASS**

Table	Expected	Actual (Oracle)	Result
APP_USER	8	8	PASS
MOVIE	12	12	PASS
GENRE	9	9	PASS
MOVIE_GENRE	28	28	PASS
ACTOR	14	14	PASS
MOVIE_ACTOR	19	19	PASS
REVIEW	26	26	PASS
WATCHLIST	15	15	PASS
THEATRE	4	4	PASS
SCREEN	7	7	PASS
SEAT	84	84	PASS
MOVIE_SHOW	12	12	PASS
BOOKING	12	12	PASS
BOOKING_SEAT	21	21	PASS
PAYMENT	12	12	PASS

Step 7 - 04_queries.sql (37 queries)

Q	Concept	Rows	Check	Result
1	Basic SELECT	12	row count cross-check	PASS
2	WHERE	2	value check vs expected	PASS
3	ORDER BY, date formatting	12	row count cross-check	PASS
4	WHERE with IS NULL and AND	4	row count cross-check	PASS
5	INNER JOIN	26	row count cross-check	PASS
6	LEFT JOIN + COUNT	12	value check vs expected	PASS
7	LEFT JOIN	9	value check vs expected	PASS
8	LEFT JOIN, finding "missing" rows	1	value check vs expected	PASS
9	Multi-table join: 4 tables	12	row count cross-check	PASS
10	Multi-table join + DISTINCT	3	value check vs expected	PASS
11	Multi-table join: 6 tables	6	row count cross-check	PASS
12	GROUP BY + AVG + HAVING	8	value check vs expected	PASS
13	GROUP BY + HAVING	4	value check vs expected	PASS
14	HAVING	4	value check vs expected	PASS
15	HAVING	1	value check vs expected	PASS
16	MAX / MIN	11	row count cross-check	PASS
17	GROUP BY + SUM + HAVING	2	value check vs expected	PASS
18	GROUP BY on two columns	2	value check vs expected	PASS
19	Subquery in HAVING	5	value check vs expected	PASS
20	Nested aggregate in subquery	4	value check vs expected	PASS
21	Nested IN subqueries	5	value check vs expected	PASS
22	Correlated subquery	6	value check vs expected	PASS
23	Correlated subquery	10	row count cross-check	PASS
24	Correlated scalar subqueries in SELECT	3	value check vs expected	PASS
25	EXISTS	7	value check vs expected	PASS
26	NOT EXISTS	1	value check vs expected	PASS
27	NOT EXISTS	5	value check vs expected	PASS
28	NOT EXISTS	6	value check vs expected	PASS
29	DOUBLE NOT EXISTS = relational division	3	value check vs expected	PASS
30	DOUBLE NOT EXISTS, second example	1	value check vs expected	PASS
31	CASE with LEFT JOIN	12	value check vs expected	PASS
32	CASE inside an aggregate	12	value check vs expected	PASS
33	UNION	4	value check vs expected	PASS
34	INTERSECT	6	value check vs expected	PASS
35	MINUS	1	value check vs expected	PASS
36	MINUS	1	value check vs expected	PASS
37	INTERSECT	7	value check vs expected	PASS

Q1 [Basic SELECT] List every movie with its language and running time.

```
SELECT title, movie_language, duration_min
FROM   movie;
```

TITLE	MOVIE_LANGUAGE	DURATION_MIN
Inception	English	148
Interstellar	English	169
The Dark Knight	English	152
Oppenheimer	English	180
Parasite	Korean	132
Spirited Away	Japanese	125
3 Idiots	Hindi	170
Dangal	Hindi	161
Andhadhun	Hindi	139
Baahubali 2: The Conclusion	Telugu	167
RRR	Telugu	187
Eega	Telugu	134

12 row(s) | **PASS**

Q2 [WHERE] Telugu movies longer than 2.5 hours.

```
SELECT title, duration_min
FROM   movie
WHERE  movie_language = 'Telugu'
AND    duration_min > 150;
```

Expected: Baahubali 2, RRR

TITLE	DURATION_MIN
Baahubali 2: The Conclusion	167
RRR	187

2 row(s) | **PASS**

Q3 [ORDER BY, date formatting] Movies from newest to oldest.

```
SELECT title, TO_CHAR(release_date, 'DD-Mon-YYYY') AS released
FROM   movie
ORDER  BY release_date DESC;
```

TITLE	RELEASED
Oppenheimer	21-Jul-2023
RRR	25-Mar-2022
Parasite	30-May-2019
Andhadhun	05-Oct-2018
Baahubali 2: The Conclusion	28-Apr-2017
Dangal	23-Dec-2016
Interstellar	07-Nov-2014
Eega	06-Jul-2012
Inception	16-Jul-2010
3 Idiots	25-Dec-2009
The Dark Knight	18-Jul-2008
Spirited Away	20-Jul-2001

12 row(s) | **PASS**

Q4 [WHERE with IS NULL and AND] Rating-only logs

```
SELECT user_id, movie_id, rating
FROM   review
WHERE  review_text IS NULL
AND    rating >= 4;
```

USER_ID	MOVIE_ID	RATING
1	4	4
2	8	4
4	9	4
7	3	4

4 row(s) | **PASS**

Q5 [INNER JOIN] Show every review with the reviewer's username and the movie title.

```
SELECT u.username, m.title, r.rating, r.review_text
FROM   review r
JOIN   app_user u ON u.user_id  = r.user_id
JOIN   movie    m ON m.movie_id = r.movie_id
ORDER  BY m.title, r.rating DESC;
```

USERNAME	TITLE	RATING	REVIEW_TEXT
ananya_m	3 Idiots	5	All izz well!
priya_s	3 Idiots	4.5	Funny and meaningful.
priya_s	Andhadhun	4.5	The twists keep coming.
sneha_r	Andhadhun	4	
karthik_n	Baahubali 2: The Conclusion	4	Grand visuals.
rahul_v	Baahubali 2: The Conclusion	3	
rahul_v	Dangal	4.5	Inspiring sports drama.
priya_s	Dangal	4	
aarav_k	Inception	5	Mind-bending from start to finish.
priya_s	Inception	4	Great concept, a little confusing.
karthik_n	Inception	3.5	
priya_s	Interstellar	5	Made me cry in a cinema hall.

USERNAME	TITLE	RATING	REVIEW_TEXT
aarav_k	Interstellar	4.5	The docking scene is unforgettable.
karthik_n	Interstellar	4	Good science, slow middle.
aarav_k	Oppenheimer	4	
rahul_v	Oppenheimer	3.5	Brilliant but too long.
sneha_r	Parasite	5	A masterpiece about class.
vikram_j	Parasite	4.5	Genre-bending and sharp.
karthik_n	RRR	5	Pure cinema.
aarav_k	RRR	4.5	Naatu Naatu alone is worth the ticket.
vikram_j	RRR	2.5	Too over the top for me.
sneha_r	Spirited Away	5	Beautiful animation.
ananya_m	Spirited Away	4.5	Still magical on a rewatch.
aarav_k	The Dark Knight	5	Best superhero film ever made.
rahul_v	The Dark Knight	4.5	Heath Ledger is incredible.
vikram_j	The Dark Knight	4	

26 row(s) | **PASS**

Q6 [LEFT JOIN + COUNT] Number of reviews for every movie, including movies with zero reviews.

```
SELECT m.title, COUNT(r.user_id) AS review_count
FROM   movie m
LEFT   JOIN review r ON r.movie_id = m.movie_id
GROUP  BY m.movie_id, m.title
ORDER  BY review_count DESC, m.title;
```

Expected: Eega shows 0.

TITLE	REVIEW_COUNT
Inception	3
Interstellar	3
RRR	3
The Dark Knight	3
3 Idiots	2
Andhadhun	2
Baahubali 2: The Conclusion	2
Dangal	2
Oppenheimer	2
Parasite	2
Spirited Away	2
Eega	0

12 row(s) | **PASS**

Q7 [LEFT JOIN] Number of movies in each genre, including empty genres.

```
SELECT g.genre_name, COUNT(mg.movie_id) AS movie_count
FROM   genre g
LEFT   JOIN movie_genre mg ON mg.genre_id = g.genre_id
GROUP  BY g.genre_name
ORDER  BY movie_count DESC;
```

Expected: Drama 8, Action 5, Thriller 4, ... Romance 0

GENRE_NAME	MOVIE_COUNT
Drama	8
Action	5
Thriller	4
Comedy	3
Fantasy	3
Sci-Fi	2
Biography	2
Animation	1
Romance	0

9 row(s) | **PASS**

Q8 [LEFT JOIN, finding "missing" rows] Movies with no cast recorded.

```
SELECT m.title
FROM   movie m
LEFT   JOIN movie_actor ma ON ma.movie_id = m.movie_id
WHERE  ma.movie_id IS NULL;
```

Expected: Spirited Away

TITLE
Spirited Away

1 row(s) | **PASS**

Q9 [Multi-table join: 4 tables] Full show schedule.

```
SELECT t.theatre_name, t.city, sc.screen_no, sc.screen_type, m.title,
       TO_CHAR(sh.show_start, 'DD-Mon HH24:MI') AS show_time, sh.base_price
FROM   movie_show sh
JOIN   movie   m  ON m.movie_id    = sh.movie_id
JOIN   screen  sc ON sc.screen_id  = sh.screen_id
JOIN   theatre t  ON t.theatre_id  = sc.theatre_id
ORDER  BY sh.show_start;
```

THEATRE_NAME	CITY	SCREEN_NO	SCREEN_TYPE	TITLE	SHOW_TIME	BASE_PRICE
Cineplex Central	Hyderabad	1	IMAX	RRR	02-Oct 18:30	250
Starlight Multiplex	Hyderabad	1	2D	RRR	02-Oct 21:00	200
Cineplex Central	Hyderabad	1	IMAX	Inception	03-Oct 14:00	300
Cineplex Central	Hyderabad	1	IMAX	Interstellar	03-Oct 18:00	300
Cineplex Central	Hyderabad	2	2D	Interstellar	03-Oct 18:00	200
Cineplex Central	Hyderabad	3	3D	Baahubali 2: The Conclusion	04-Oct 11:00	180
Starlight Multiplex	Hyderabad	2	2D	Oppenheimer	04-Oct 15:00	220
Royal Talkies	Hyderabad	1	2D	3 Idiots	04-Oct 19:00	150
Royal Talkies	Hyderabad	1	2D	Spirited Away	05-Oct 10:00	150
Cineplex Central	Hyderabad	2	2D	Andhadhun	05-Oct 13:00	180
Orion Screens	Bengaluru	1	2D	RRR	05-Oct 20:00	220
Starlight Multiplex	Hyderabad	1	2D	Eega	06-Oct 12:00	160

12 row(s) | **PASS**

Q10 [Multi-table join + DISTINCT] Theatres showing RRR.

```
SELECT DISTINCT t.theatre_name, t.city
FROM   theatre t
JOIN   screen     sc ON sc.theatre_id = t.theatre_id
JOIN   movie_show sh ON sh.screen_id  = sc.screen_id
JOIN   movie      m  ON m.movie_id    = sh.movie_id
WHERE  m.title = 'RRR';
```

Expected: Cineplex Central, Starlight Multiplex, Orion Screens

THEATRE_NAME	CITY
Cineplex Central	Hyderabad
Starlight Multiplex	Hyderabad
Orion Screens	Bengaluru

3 row(s) | **PASS**

Q11 [Multi-table join: 6 tables] Ticket details of one user

```
SELECT b.booking_id, m.title, t.theatre_name,
       TO_CHAR(sh.show_start, 'DD-Mon HH24:MI') AS show_time,
       st.seat_row || st.seat_number AS seat, bs.price_paid
FROM   app_user u
JOIN   booking      b  ON b.user_id     = u.user_id
JOIN   booking_seat bs ON bs.booking_id = b.booking_id
JOIN   seat         st ON st.seat_id    = bs.seat_id
JOIN   movie_show   sh ON sh.show_id    = b.show_id
JOIN   movie        m  ON m.movie_id    = sh.movie_id
JOIN   screen       sc ON sc.screen_id  = sh.screen_id
JOIN   theatre      t  ON t.theatre_id  = sc.theatre_id
WHERE  u.username = 'aarav_k'
ORDER  BY b.booking_id, seat;
```

BOOKING_ID	TITLE	THEATRE_NAME	SHOW_TIME	SEAT	PRICE_PAID
1	RRR	Cineplex Central	02-Oct 18:30	B1	250
1	RRR	Cineplex Central	02-Oct 18:30	B2	250

BOOKING_ID	TITLE	THEATRE_NAME	SHOW_TIME	SEAT	PRICE_PAID
8	Baahubali 2: The Conclusion	Cineplex Central	04-Oct 11:00	B1	180
8	Baahubali 2: The Conclusion	Cineplex Central	04-Oct 11:00	B2	180
8	Baahubali 2: The Conclusion	Cineplex Central	04-Oct 11:00	B3	180
8	Baahubali 2: The Conclusion	Cineplex Central	04-Oct 11:00	B4	180

6 row(s) | **PASS**

Q12 [GROUP BY + AVG + HAVING] Movies with average rating above 4.

```
SELECT m.title, ROUND(AVG(r.rating), 2) AS avg_rating, COUNT(*) AS reviews
FROM   movie m
JOIN   review r ON r.movie_id = m.movie_id
GROUP  BY m.movie_id, m.title
HAVING AVG(r.rating) > 4
ORDER  BY avg_rating DESC;
```

Expected: 8 movies. RRR (exactly 4.0) is NOT included.

TITLE	AVG_RATING	REVIEWS
3 Idiots	4.75	2
Parasite	4.75	2
Spirited Away	4.75	2
The Dark Knight	4.5	3
Interstellar	4.5	3
Andhadhun	4.25	2
Dangal	4.25	2
Inception	4.17	3

8 row(s) | **PASS**

Q13 [GROUP BY + HAVING] Movies belonging to more than two genres.

```
SELECT m.title, COUNT(*) AS genre_count
FROM   movie m
JOIN   movie_genre mg ON mg.movie_id = m.movie_id
GROUP  BY m.movie_id, m.title
HAVING COUNT(*) > 2;
```

Expected: Inception, The Dark Knight, Parasite, Baahubali 2

TITLE	GENRE_COUNT
Inception	3
The Dark Knight	3
Parasite	3
Baahubali 2: The Conclusion	3

4 row(s) | **PASS**

Q14 [HAVING] Users who have reviewed more than 3 movies.

```
SELECT u.username, COUNT(*) AS reviews_written
FROM   app_user u
JOIN   review r ON r.user_id = u.user_id
GROUP  BY u.user_id, u.username
HAVING COUNT(*) > 3
ORDER  BY reviews_written DESC;
```

Expected: aarav_k 5, priya_s 5, rahul_v 4, karthik_n 4
Note: rahul_v and karthik_n tie on 4 reviews, so their relative order is not guaranteed.

USERNAME	REVIEWS_WRITTEN
aarav_k	5
priya_s	5
rahul_v	4
karthik_n	4

4 row(s) | **PASS**

Q15 [HAVING] Theatres having more than 2 screens.

```
SELECT t.theatre_name, COUNT(*) AS screens
FROM   theatre t
JOIN   screen sc ON sc.theatre_id = t.theatre_id
GROUP  BY t.theatre_id, t.theatre_name
HAVING COUNT(*) > 2;
```

Expected: Cineplex Central (3)

THEATRE_NAME	SCREENS
Cineplex Central	3

1 row(s) | **PASS**

Q16 [MAX / MIN] Highest and lowest rating each movie received.

```
SELECT m.title, MAX(r.rating) AS highest, MIN(r.rating) AS lowest,
       MAX(r.rating) - MIN(r.rating) AS spread
FROM   movie m
JOIN   review r ON r.movie_id = m.movie_id
GROUP  BY m.movie_id, m.title
ORDER  BY spread DESC;
```

TITLE	HIGHEST	LOWEST	SPREAD
RRR	5	2.5	2.5
Inception	5	3.5	1.5
Baahubali 2: The Conclusion	4	3	1
The Dark Knight	5	4	1
Interstellar	5	4	1
Parasite	5	4.5	.5
Andhadhun	4.5	4	.5
Dangal	4.5	4	.5
Spirited Away	5	4.5	.5
Oppenheimer	4	3.5	.5
3 Idiots	5	4.5	.5

11 row(s) | **PASS**

Q17 [GROUP BY + SUM + HAVING] Users whose confirmed bookings total more than Rs 800.

```
SELECT u.username, SUM(bs.price_paid) AS total_spent
FROM   app_user u
JOIN   booking      b  ON b.user_id     = u.user_id
JOIN   booking_seat bs ON bs.booking_id = b.booking_id
WHERE  b.status = 'CONFIRMED'
GROUP  BY u.user_id, u.username
HAVING SUM(bs.price_paid) > 800
ORDER  BY total_spent DESC;
```

Expected: aarav_k 1220, rahul_v 1180

USERNAME	TOTAL_SPENT
aarav_k	1220
rahul_v	1180

2 row(s) | **PASS**

Q18 [GROUP BY on two columns] Actors who appear in more than 2 movies.

```
SELECT a.actor_name, COUNT(*) AS movies
FROM   actor a
JOIN   movie_actor ma ON ma.actor_id = a.actor_id
GROUP  BY a.actor_id, a.actor_name
HAVING COUNT(*) > 2;
```

Expected: Cillian Murphy 3, Michael Caine 3

ACTOR_NAME	MOVIES
Cillian Murphy	3
Michael Caine	3

2 row(s) | **PASS**

Q19 [Subquery in HAVING] Movies rated above the overall average rating of all reviews

```
SELECT m.title, ROUND(AVG(r.rating), 2) AS avg_rating
FROM   movie m
JOIN   review r ON r.movie_id = m.movie_id
GROUP  BY m.movie_id, m.title
HAVING AVG(r.rating) > (SELECT AVG(rating) FROM review);
```

Expected: Interstellar, The Dark Knight, Parasite, Spirited Away, 3 Idiots

TITLE	AVG_RATING
Interstellar	4.5
The Dark Knight	4.5
3 Idiots	4.75
Parasite	4.75
Spirited Away	4.75

5 row(s) | **PASS**

Q20 [Nested aggregate in subquery] The most reviewed movie

```
SELECT m.title, COUNT(*) AS review_count
FROM   movie m
JOIN   review r ON r.movie_id = m.movie_id
GROUP  BY m.movie_id, m.title
HAVING COUNT(*) = (SELECT MAX(COUNT(*)) FROM review GROUP BY movie_id);
```

Expected: Inception, Interstellar, The Dark Knight, RRR (3 reviews each)

TITLE	REVIEW_COUNT
Inception	3
Interstellar	3
The Dark Knight	3
RRR	3

4 row(s) | **PASS**

Q21 [Nested IN subqueries] Users who have booked at Cineplex Central

```
SELECT username
FROM   app_user
WHERE  user_id IN (SELECT b.user_id
                   FROM   booking b
                   WHERE  b.status = 'CONFIRMED'
                   AND    b.show_id IN (SELECT sh.show_id
                                        FROM   movie_show sh
                                        JOIN   screen  sc ON sc.screen_id = sh.screen_id
                                        JOIN   theatre t  ON t.theatre_id = sc.theatre_id
                                        WHERE  t.theatre_name = 'Cineplex Central'));
```

Expected: aarav_k, priya_s, rahul_v, sneha_r, vikram_j

USERNAME
aarav_k
priya_s
rahul_v
sneha_r
vikram_j

5 row(s) | **PASS**

Q22 [Correlated subquery] Movies longer than the average length of movies in the SAME language.

```
SELECT m.title, m.movie_language, m.duration_min
FROM   movie m
WHERE  m.duration_min > (SELECT AVG(m2.duration_min)
                         FROM   movie m2
                         WHERE  m2.movie_language = m.movie_language);
```

Expected: Interstellar, Oppenheimer, 3 Idiots, Dangal, Baahubali 2, RRR

TITLE	MOVIE_LANGUAGE	DURATION_MIN
Interstellar	English	169
Oppenheimer	English	180
3 Idiots	Hindi	170
Dangal	Hindi	161
Baahubali 2: The Conclusion	Telugu	167
RRR	Telugu	187

6 row(s) | **PASS**

Q23 [Correlated subquery] Each user's highest-rated movie

```
SELECT u.username, m.title, r.rating
FROM   review r
JOIN   app_user u ON u.user_id  = r.user_id
JOIN   movie    m ON m.movie_id = r.movie_id
WHERE  r.rating = (SELECT MAX(r2.rating)
                   FROM   review r2
                   WHERE  r2.user_id = r.user_id)
ORDER  BY u.username;
```

USERNAME	TITLE	RATING
aarav_k	The Dark Knight	5
aarav_k	Inception	5
ananya_m	3 Idiots	5
karthik_n	RRR	5
priya_s	Interstellar	5
rahul_v	The Dark Knight	4.5
rahul_v	Dangal	4.5
sneha_r	Spirited Away	5
sneha_r	Parasite	5
vikram_j	Parasite	4.5

10 row(s) | **PASS**

Q24 [Correlated scalar subqueries in SELECT] Upcoming RRR shows with the number of seats still available.

```
SELECT sh.show_id, t.theatre_name,
       TO_CHAR(sh.show_start, 'DD-Mon HH24:MI') AS show_time,
       (SELECT COUNT(*) FROM seat st WHERE st.screen_id = sh.screen_id)
     - (SELECT COUNT(*) FROM booking_seat bs WHERE bs.show_id = sh.show_id) AS seats_left
FROM   movie_show sh
JOIN   movie   m  ON m.movie_id   = sh.movie_id
JOIN   screen  sc ON sc.screen_id = sh.screen_id
JOIN   theatre t  ON t.theatre_id = sc.theatre_id
WHERE  m.title = 'RRR'
ORDER  BY sh.show_start;
```

SHOW_ID	THEATRE_NAME	SHOW_TIME	SEATS_LEFT
1	Cineplex Central	02-Oct 18:30	6
2	Starlight Multiplex	02-Oct 21:00	11
9	Orion Screens	05-Oct 20:00	10

3 row(s) | **PASS**

Q25 [EXISTS] Users who have reviewed at least one movie.

```
SELECT u.username
FROM   app_user u
WHERE  EXISTS (SELECT 1 FROM review r WHERE r.user_id = u.user_id);
```

Expected: users 1-7 (not divya_p)

USERNAME
aarav_k
priya_s
rahul_v
sneha_r
karthik_n
ananya_m
vikram_j

7 row(s) | **PASS**

Q26 [NOT EXISTS] Movies that have never been reviewed.

```
SELECT m.title
FROM   movie m
WHERE  NOT EXISTS (SELECT 1 FROM review r WHERE r.movie_id = m.movie_id);
```

Expected: Eega

TITLE
Eega

1 row(s) | **PASS**

Q27 [NOT EXISTS] Movies with no bookings at all

```
SELECT m.title
FROM   movie m
WHERE  NOT EXISTS (SELECT 1
                   FROM   movie_show sh
                   JOIN   booking b ON b.show_id = sh.show_id
                   WHERE  sh.movie_id = m.movie_id);
```

Expected: The Dark Knight, Parasite, Spirited Away, Dangal, Eega

TITLE
The Dark Knight
Parasite
Spirited Away
Dangal
Eega

5 row(s) | **PASS**

Q28 [NOT EXISTS] Available seats for show 1

```
SELECT st.seat_row || st.seat_number AS seat, st.seat_type
FROM   movie_show sh
JOIN   seat st ON st.screen_id = sh.screen_id
WHERE  sh.show_id = 1
AND    NOT EXISTS (SELECT 1
                   FROM   booking_seat bs
                   WHERE  bs.show_id = sh.show_id
                   AND    bs.seat_id = st.seat_id)
ORDER  BY st.seat_row, st.seat_number;
```

Expected: A1, A2, A3, A4, C3, C4

SEAT	SEAT_TYPE
A1	REGULAR
A2	REGULAR
A3	REGULAR
A4	REGULAR
C3	PREMIUM
C4	PREMIUM

6 row(s) | **PASS**

Q29 [DOUBLE NOT EXISTS = relational division] Users who have reviewed EVERY Sci-Fi movie.

```
SELECT u.username
FROM   app_user u
WHERE  NOT EXISTS (
         SELECT 1
         FROM   movie_genre mg
         JOIN   genre g ON g.genre_id = mg.genre_id
         WHERE  g.genre_name = 'Sci-Fi'
         AND    NOT EXISTS (SELECT 1
                            FROM   review r
                            WHERE  r.user_id  = u.user_id
                            AND    r.movie_id = mg.movie_id));
```

Expected: aarav_k, priya_s, karthik_n
Note: Relational division: correct users returned.

USERNAME
aarav_k
priya_s
karthik_n

3 row(s) | **PASS**

Q30 [DOUBLE NOT EXISTS, second example] Theatres that are showing EVERY Sci-Fi movie.

```
SELECT t.theatre_name
FROM   theatre t
WHERE  NOT EXISTS (
         SELECT 1
         FROM   movie_genre mg
         JOIN   genre g ON g.genre_id = mg.genre_id
         WHERE  g.genre_name = 'Sci-Fi'
         AND    NOT EXISTS (SELECT 1
                            FROM   movie_show sh
                            JOIN   screen sc ON sc.screen_id = sh.screen_id
                            WHERE  sc.theatre_id = t.theatre_id
                            AND    sh.movie_id   = mg.movie_id));
```

Expected: Cineplex Central (shows both Inception and Interstellar)

THEATRE_NAME
Cineplex Central

1 row(s) | **PASS**

Q31 [CASE with LEFT JOIN] Label each movie by its average rating.

```
SELECT m.title,
       ROUND(AVG(r.rating), 2) AS avg_rating,
       CASE
         WHEN AVG(r.rating) IS NULL THEN 'Not rated yet'
         WHEN AVG(r.rating) >= 4.5  THEN 'Must watch'
         WHEN AVG(r.rating) >= 3.5  THEN 'Good'
         ELSE                            'Mixed reviews'
       END AS verdict
FROM   movie m
LEFT   JOIN review r ON r.movie_id = m.movie_id
GROUP  BY m.movie_id, m.title
ORDER  BY avg_rating DESC NULLS LAST;
```

TITLE	AVG_RATING	VERDICT
3 Idiots	4.75	Must watch
Spirited Away	4.75	Must watch
Parasite	4.75	Must watch
Interstellar	4.5	Must watch
The Dark Knight	4.5	Must watch
Andhadhun	4.25	Good
Dangal	4.25	Good
Inception	4.17	Good
RRR	4	Good
Oppenheimer	3.75	Good
Baahubali 2: The Conclusion	3.5	Good
Eega		Not rated yet

12 row(s) | **PASS**

Q32 [CASE inside an aggregate] Occupancy of every show, with a label.

```
SELECT sh.show_id, m.title,
       COUNT(bs.seat_id) AS booked,
       (SELECT COUNT(*) FROM seat st WHERE st.screen_id = sh.screen_id) AS capacity,
       SUM(CASE WHEN st2.seat_type = 'PREMIUM' THEN 1 ELSE 0 END) AS premium_booked,
       CASE
         WHEN COUNT(bs.seat_id) = 0 THEN 'No bookings'
         WHEN COUNT(bs.seat_id) >= 6 THEN 'Filling fast'
         ELSE 'Available'
       END AS status_label
FROM   movie_show sh
JOIN   movie m ON m.movie_id = sh.movie_id
LEFT   JOIN booking_seat bs ON bs.show_id = sh.show_id
LEFT   JOIN seat st2        ON st2.seat_id = bs.seat_id
GROUP  BY sh.show_id, sh.screen_id, m.title
ORDER  BY sh.show_id;
```

SHOW_ID	TITLE	BOOKED	CAPACITY	PREMIUM_BOOKED	STATUS_LABEL
1	RRR	6	12	2	Filling fast
2	RRR	1	12	1	Available
3	Inception	2	12	2	Available
4	Interstellar	3	12	0	Available
5	Interstellar	0	12	0	No bookings

SHOW_ID	TITLE	BOOKED	CAPACITY	PREMIUM_BOOKED	STATUS_LABEL
6	Baahubali 2: The Conclusion	4	12	0	Available
7	Oppenheimer	0	12	0	No bookings
8	3 Idiots	2	12	0	Available
9	RRR	2	12	0	Available
10	Andhadhun	1	12	1	Available
11	Spirited Away	0	12	0	No bookings
12	Eega	0	12	0	No bookings

12 row(s) | **PASS**

Q33 [UNION] Every city the platform is active in

```
SELECT city FROM app_user WHERE city IS NOT NULL
UNION
SELECT city FROM theatre
ORDER  BY 1;
```

Expected: Bengaluru, Chennai, Hyderabad, Mumbai

CITY
Bengaluru
Chennai
Hyderabad
Mumbai

4 row(s) | **PASS**

Q34 [INTERSECT] Users who have both reviewed and booked.

```
SELECT u.username FROM app_user u JOIN review  r ON r.user_id = u.user_id
INTERSECT
SELECT u.username FROM app_user u JOIN booking b ON b.user_id = u.user_id;
```

Expected: aarav_k, priya_s, rahul_v, sneha_r, karthik_n, vikram_j

USERNAME
aarav_k
priya_s
rahul_v
sneha_r
karthik_n
vikram_j

6 row(s) | **PASS**

Q35 [MINUS] Users who review movies but have never booked a ticket.

```
SELECT user_id FROM review
MINUS
SELECT user_id FROM booking;
```

Expected: ananya_m
Note: The comment says 'ananya_m' but the query returns user_id 6 (which is ananya_m).

USER_ID
6

1 row(s) | **PASS**

Q36 [MINUS] Users who book tickets but have never written a review.

```
SELECT user_id FROM booking
MINUS
SELECT user_id FROM review;
```

Expected: 8 (divya_p)

USER_ID
8

1 row(s) | **PASS**

Q37 [INTERSECT] Watchlisted movies that are currently scheduled

```
SELECT movie_id FROM watchlist
INTERSECT
SELECT movie_id FROM movie_show
ORDER  BY 1;
```

Expected: movie_ids 1, 2, 4, 6, 10, 11, 12

MOVIE_ID
1
2
4
6
10
11
12

7 row(s) | **PASS**

Step 8 - 05_views.sql

5 views created: MOVIE_RATING_SUMMARY, MOVIE_DETAILS, THEATRE_SHOW_SCHEDULE, BOOKING_SUMMARY, SHOW_SEAT_STATUS. Demo queries on the views are shown below. Result: **PASS**

Top-rated movies with at least 2 reviews

TITLE	AVG_RATING	REVIEW_COUNT
3 Idiots	4.75	2
Parasite	4.75	2
Spirited Away	4.75	2
The Dark Knight	4.5	3
Interstellar	4.5	3
Dangal	4.25	2
Andhadhun	4.25	2
Inception	4.17	3
RRR	4	3
Oppenheimer	3.75	2
Baahubali 2: The Conclusion	3.5	2

PASS

A movie page (Inception)

MOVIE_ID	TITLE	RELEASE_DATE	DURATION_MIN	MOVIE_LANGUAGE	CERTIFICATE	GENRES	CAST_LIST
1	Inception	2010-07-16 00:00	148	English	UA	Action, Sci-Fi, Thriller	Cillian Murphy, Leonardo DiCaprio, Michael Caine

PASS

Everything playing in Hyderabad on 3 October

THEATRE_NAME	SCREEN_NO	TITLE	STARTS	BASE_PRICE
Cineplex Central	1	Inception	14:00	300
Cineplex Central	2	Interstellar	18:00	200
Cineplex Central	1	Interstellar	18:00	300

PASS

Bookings whose charge differs from seats (cancelled only)

BOOKING_ID	USERNAME	BOOKING_STATUS	TOTAL_AMOUNT	AMOUNT_CHARGED	PAYMENT_STATUS
10	divya_p	CANCELLED	0	400	FAILED
9	priya_s	CANCELLED	0	220	REFUNDED

PASS

Seat map for show 1

SEAT_ROW	SEATS
A	A1 A2 A3 A4
B	B1(X) B2(X) B3(X) B4(X)
C	C1(X) C2(X) C3 C4

PASS

Views in the data dictionary

VIEW_NAME
BOOKING_SUMMARY
MOVIE_DETAILS
MOVIE_RATING_SUMMARY
SHOW_SEAT_STATUS
THEATRE_SHOW_SCHEDULE

PASS

Step 9 - 06_booking_integrity.sql

The trigger TRG_BOOKING_SEAT_SCREEN was created. Then the test script ran in one transaction: 11 invalid operations, each of which must be rejected with a specific error, plus valid operations and the SAVEPOINT / ROLLBACK demo. Result: **PASS**

#	Statement	Expected	Actual (Oracle)	Result
1	CREATE OR REPLACE TRIGGER trg_booking_seat_screen	succeeds	Trigger created.	PASS
2	INSERT INTO booking VALUES (900, 8, 1, 'PENDING');	succeeds	1 row created.	PASS
3	INSERT INTO booking_seat VALUES (900, 1, 101, 250) ;	succeeds	1 row created.	PASS
4	SAVEPOINT after_valid_seat;	succeeds	Savepoint created.	PASS
5	INSERT INTO booking VALUES (901, 8, 999, 'PENDING');	ORA-02291 (FK_BOOKING_SHOW)	ORA-02291: integrity constraint (FK_BOOKING_SHOW) violated - parent key not found	PASS
6	INSERT INTO booking_seat VALUES (900, 1, 9999, 250);	ORA-02291 (FK_BSEAT_SEAT)	ORA-02291: integrity constraint (FK_BSEAT_SEAT) violated - parent key not found	PASS
7	INSERT INTO booking_seat VALUES (900, 1, 105, 250);	ORA-00001 (UQ_BSEAT_SHOW_SEAT)	ORA-00001: unique constraint (UQ_BSEAT_SHOW_SEAT) violated	PASS
8	INSERT INTO booking_seat VALUES (900, 1, 201, 250);	ORA-20001	ORA-20001: Seat 201 is not in screen 1 where show 1 is running.	PASS
9	INSERT INTO booking_seat VALUES (900, 2, 401, 200);	ORA-02291 (FK_BSEAT_BOOKING)	ORA-02291: integrity constraint (FK_BSEAT_BOOKING) violated - parent key not found	PASS
10	INSERT INTO booking VALUES (902, 8, 1, 'DONE');	ORA-02290 (CK_BOOKING_STATUS)	ORA-02290: check constraint (CK_BOOKING_STATUS) violated	PASS
11	UPDATE booking SET status = 'BOOKED' WHERE booking_id = 900;	ORA-02290 (CK_BOOKING_STATUS)	ORA-02290: check constraint (CK_BOOKING_STATUS) violated	PASS
12	INSERT INTO payment VALUES (900, 900, 250, 'UPI', 'PAID', SYSDATE);	ORA-02290 (CK_PAY_STATUS)	ORA-02290: check constraint (CK_PAY_STATUS) violated	PASS
13	INSERT INTO payment VALUES (900, 900, 250, 'CASH', 'SUCCESS', ...	ORA-02290 (CK_PAY_METHOD)	ORA-02290: check constraint (CK_PAY_METHOD) violated	PASS
14	INSERT INTO payment VALUES (900, 900, 250, 'UPI', 'SUCCESS');	ORA-02290 (CK_PAY_PAID_AT)	ORA-02290: check constraint (CK_PAY_PAID_AT) violated	PASS
15	INSERT INTO payment VALUES (900, 900, 250, 'UPI', 'SUCCESS', ...	succeeds	1 row created.	PASS
16	UPDATE booking SET status = 'CONFIRMED' WHERE booking_id = 900 ;	succeeds	1 row updated.	PASS
17	INSERT INTO payment VALUES (901, 900, 250, 'CARD', 'SUCCESS', ...	ORA-00001 (UQ_PAYMENT_BOOKING)	ORA-00001: unique constraint (UQ_PAYMENT_BOOKING) violated	PASS
18	SELECT seat_label, seat_status FROM show_seat_status WHERE show_id ...	A1 BOOKED, A2-A4 AVAILABLE	A1 BOOKED, A2 AVAILABLE, A3 AVAILABLE, A4 AVAILABLE	PASS
19	ROLLBACK TO SAVEPOINT after_valid_seat;	succeeds	Rollback complete.	PASS
20	SELECT booking_id, status FROM booking WHERE booking_id = 900 ;	900 PENDING	900 PENDING	PASS
21	SELECT COUNT(*) AS payments_for_900 FROM payment WHERE booking_id ...	PAYMENTS_FOR_900 = 0	PAYMENTS_FOR_900 = 0	PASS
22	ROLLBACK;	succeeds	Rollback complete.	PASS
23	SELECT COUNT(*) AS booking_900_rows FROM booking WHERE booking_id ...	BOOKING_900_ROWS = 0	BOOKING_900_ROWS = 0	PASS

Everything was rolled back at the end, so the sample data is unchanged after the test.

How to reproduce

From the MovieDB_Oracle folder: `python report\run_and_capture.py` then `python report\build_report.py`. Or, in SQL Developer, connect to XEPDB1 as MOVIEDB (see oracle_credentials.txt) and run 02 to 06 in order with Run Script (F5).